Sistema de Campanhas WhatsApp

Avaliação de pontos fortes e fracos

Radrasys — módulo Campanhas WhatsApp (aba Inteligência de Clientes › Crescimento e Resultado › Marketing, Clientes e Receita). Documento gerado em 17/10/2026 a partir do estado real do código, da documentação viva do repositório (docs/campanhas-whatsapp.md, docs/campanhas-whatsapp-ia.md, AGENTS.md) e da suíte de testes automatizados (152 testes verificados nesta avaliação, cobrindo regras de cadência/quarentena/pós-venda, webhooks REST, Fontes de Dados do ERP e qualificação de leads por IA).

Resumo executivo

O módulo cobre bem o ciclo operacional de uma campanha de WhatsApp B2B: definir a audiência (combinando públicos automáticos do ERP com listas externas), respeitar limites de frequência e anti-spam, disparar pelo canal oficial e, desde 17/10/2026, qualificar semanticamente as respostas com IA. A base de regra de negócio é pura e testada, e dois achados reais de produção/teste (disparo em paralelo não entregava; o modelo de IA truncava a resposta) já foram corrigidos e documentados. A lacuna mais relevante não é de código: o canal de mensagens recebidas (webhook) ainda não tem tráfego real porque depende de uma configuração do lado da Ceci.Chat, fora deste repositório.

Pontos fortes

1. Regras de negócio centrais são puras, testadas e já passaram por achados reais

- **Cadência, quarentena e pós-venda bem modelados**
  Semáforo de prazo (vermelho/amarelo/verde), quarentena por telefone normalizado ("recebeu de QUALQUER campanha há menos de N dias") e uma segunda trava mais fina por campanha específica (campanhas_whatsapp_contatos_historico). Pós-venda calcula o prazo pelo maior entre faturamento+frequência e um piso de 16 dias úteis (sem feriados nacionais) após a aprovação — evita contatar o cliente cedo demais mesmo que o faturamento atrase.

- **Suíte de testes real, não só unitária isolada**
  campanhas-whatsapp-db.test.ts roda contra banco Postgres real (não mockado): quarentena, registro retroativo, modo webhook, categorias, Fontes de Dados de ponta a ponta, Planner. No total, 152 testes passam hoje cobrindo o módulo inteiro (regras puras + banco real + IA + parser de lista de contatos).

- **Dois achados reais de produção já corrigidos e documentados**
  (1) 16/10/2026: disparo em paralelo (8 chamadas simultâneas) era aceito pela Ceci com HTTP 200 mas NÃO chegava ao WhatsApp do destinatário — corrigido com envio sequencial (1 por vez, intervalo de 1,2s). (2) 17/10/2026: o Gemini truncava a resposta JSON por gastar o teto de tokens em "pensamento" invisível — corrigido desligando esse recurso na chamada. Os dois ficaram registrados no código e na documentação, não só resolvidos silenciosamente.

2. Fontes de audiência flexíveis e baratas

- **10 fontes automáticas do ERP sem custo de API**
  Clientes ativos, inativos, primeira compra, reativados do mês, redução de volume etc. são calculados a partir do histórico já replicado localmente (historico_os/historico_orcamentos) — nenhuma chamada à API MubiSys para montar a lista. Combináveis entre si e com listas externas (upload) na mesma campanha.

- **Telefone tratado com cuidado acima da média**
  normalizarTelefone distingue DDI 55 de DDD 55 (RS) pelo comprimento, insere o 9º dígito em celulares antigos de 8 dígitos e extrai o melhor número de campos com vários telefones juntos. Há ainda um espelho do cadastro de clientes do MubiSys (mubisys_clientes_cache) especificamente para completar telefones que o histórico local não tinha.

3. Camada de disparo com salvaguardas reais

- **Webhooks REST protegidos e com contrato documentado**
  log-send e check-quarantine (e agora whatsapp-inbound/process-buffers) exigem CAMPANHAS_API_KEY comparada em tempo constante; sem a chave configurada o servidor responde 503 em vez de abrir sem querer. O doc traz exemplos de curl prontos com o formato exato de request/response.

- **Honestidade sobre o que o status HTTP prova**
  A documentação é explícita: "o status HTTP 200/QUEUED/READ do endpoint não garante entrega — só a confirmação no aparelho do destinatário prova". Isso evita o erro comum de tratar aceite da fila como entrega confirmada.

4. Qualificação de leads por IA (novo, 17/10/2026)

- **Pipeline completo webhook -> debounce -> IA -> dashboard**
  Filtro de bot (tempo de resposta + blacklist semântica, nunca aplicado a clique de botão/lista), consolidação de mensagens picadas numa janela de 60s antes de gastar uma chamada de IA, e controle de custo (só qualifica quem ainda não tem lead na campanha).

- **Sem dependência de infraestrutura que o deploy não suporta**
  O deploy é Vercel serverless, onde uma fila em memória (Redis-like) não sobreviveria entre instâncias. O buffer de debounce foi implementado como uma coluna de janela (buffer_ate) na própria tabela Postgres, evitando acrescentar uma peça de infraestrutura nova ao projeto.

- **Validado contra a API real, não só mockado**
  2 testes de contrato chamam o Gemini de verdade (gated por env var, mesmo padrão já usado para a API do MubiSys) e confirmam classificação correta de 4 cenários de teste (orçamento direto, dúvida técnica, objeção de fornecedor, pedido de remoção).

5. Boa separação de camadas e documentação viva

- **shared (puro) / services (regra) / routers (I/O) / routes (REST)**

Funções de data, telefone e semáforo são isomórficas e sem I/O; a regra de higienização de lista e cálculo de pós-venda é pura e testável sem banco; só o router/rotas tocam Drizzle/Postgres. Isso é o que permite testar 100+ cenários sem subir infraestrutura pesada.

- **Dois documentos vivos, atualizados a cada mudança real**
  docs/campanhas-whatsapp.md (404 linhas) e o novo docs/campanhas-whatsapp-ia.md registram decisões, datas, achados e limitações — não é documentação que descreve uma intenção inicial e ficou desatualizada.

Pontos fracos e limitações conhecidas

A maioria dos itens abaixo já está documentada no próprio repositório (não são achados escondidos) — o valor deste relatório é reuni-los num só lugar com o peso relativo de cada um.

1. Lacunas de dados e cobertura

- **A quarentena só enxerga quem passou por este módulo**
  Disparos feitos pela Retenção de Clientes Novos e cliques de WhatsApp do CRM não alimentam a quarentena central — um mesmo cliente pode, em tese, ser contatado por dois módulos diferentes sem que um saiba do outro. Risco real de fadiga de contato, não só teórico.

- **Sem histórico por contato, só o último registro**
  Não há como responder "quando foi a última vez que ESTE telefone recebeu ALGO" além do contato mais recente — nem status de entrega/resposta histórico por disparo individual.

- **Telefone ausente em boa parte do histórico antigo**
  historico_os.telefone só existe desde 21/09/2026; em teste manual, 857 de 990 inativos resolvidos vieram sem telefone, mesmo o cadastro do cliente tendo o número no MubiSys. O backfill por janela ajuda mas não fecha o gap para quem sumiu há mais de 24 meses.

2. Desempenho e custo de execução

- **gerarListaDaCampanha recarrega todo o histórico a cada chamada**
  Mesmo custo de construirBaseClientes usado em outros relatórios — aceitável no volume atual, mas é o primeiro ponto a otimizar (cache de alguns minutos) se o histórico crescer muito. O próprio código já sinaliza isso como ponto de atenção, não é um problema oculto.

- **Rate limiting de aplicação desligado em produção**
  Decisão deliberada (não um esquecimento): o MemoryStore do express-rate-limit não funciona de forma confiável em ambiente serverless (Vercel) — um contador por instância reseta a cada cold start. Afeta o app inteiro, webhooks de campanhas incluídos; reativar exigiria um store distribuído (Redis) hoje ausente.

3. Sem hierarquia formal e frequências aproximadas

- **"Subcampanha" é convenção de nome, não modelo de dados**
  Campanhas como "Prospecção Google Maps — MS/PR/RS/SC" são campanhas comuns agrupadas só pelo nome; não há relacionamento formal no banco entre uma campanha "mãe" e suas variações regionais.

- **"Novos/Reativados do mês" usam frequência de 30 dias como aproximação**
  O schema não tem o conceito "todo fim de mês"; a solução hoje é o usuário agendar manualmente pelo Planner, confirmando disparado/não disparado — funcional, mas depende de disciplina manual, não é automático de verdade.

4. Qualificação por IA: lacunas específicas do módulo novo

- **Webhook de mensagens recebidas ainda sem tráfego real**
  Esta é a limitação mais importante do módulo de IA: hoje a resposta do cliente só chega na caixa de entrada da própria Ceci.Chat. Para o pipeline (filtro de bot -> debounce -> Gemini) processar mensagem real, a Ceci precisa ser configurada, no painel deles, para replicar o webhook de mensagem recebida para a URL do Radrasys — passo manual, fora deste repositório, ainda não feito.

- **Atribuição de campanha por telefone é um palpite, não uma certeza**
  Nenhum webhook do WhatsApp devolve "a qual campanha esta resposta pertence" — o sistema usa o envio mais recente daquele telefone como melhor estimativa. Se o mesmo número recebeu de duas campanhas em sequência curta, a resposta pode ser atribuída à campanha errada.

- **"Taxa de abertura real" do pedido original não foi implementada**
  A API da WTS.Chat/Ceci hoje usada não distingue confirmação de leitura de confirmação de entrega — o indicador ficaria baseado num dado que o sistema simplesmente não tem disponível.

- **Retargeting é manual (exportar + reimportar), não um clique único**
  O filtro de leads por cluster/ICP exporta uma planilha no mesmo formato aceito pelo disparo, mas não existe ainda um botão "criar campanha de retargeting" que feche esse loop automaticamente.

- **Métricas de tempo de reação só confiáveis para disparo automático (via API)**
  Para campanhas disparadas manualmente pela tela, o horário do "registro" do disparo pode ser muito depois do envio real — por isso o sistema deliberadamente NÃO calcula tempo de resposta nesses casos (ficaria

artificialmente curto e classificaria cliente real como bot). Correto, mas reduz a cobertura do heatmap/Lead Velocity para quem usa só disparo manual.

Leitura geral e prioridade sugerida

A base de regra de negócio (cadência, quarentena, pós-venda, Fontes de Dados) é o ponto mais maduro do módulo: pura, testada contra banco real e já com dois achados de produção corrigidos em vez de ignorados. O risco maior não está no código já escrito, e sim em três frentes:

1. **Prioridade alta — fechar o laço do webhook de IA**
   Sem a Ceci configurada para enviar mensagens recebidas ao Radrasys, todo o investimento em debounce, filtro de bot e qualificação por Gemini fica sem dado real para processar. É um passo de configuração externa, não uma tarefa de código, mas é o bloqueio que mais limita o valor já construído.

2. **Prioridade média — unificar a quarentena entre módulos**
   Retenção de Clientes Novos e CRM disparando WhatsApp sem alimentar a mesma trava central é o tipo de lacuna que não aparece em teste isolado, mas pode gerar reclamação real de cliente contatado demais.

3. **Prioridade baixa — otimizações de custo/performance**
   Cache de gerarListaDaCampanha e reativação de rate limiting com um store distribuído são melhorias sensatas, mas o próprio projeto já trata as duas como "aceitável no volume atual" — não bloqueiam o uso diário do módulo hoje.

Fontes: AGENTS.md, docs/campanhas-whatsapp.md, docs/campanhas-whatsapp-ia.md e a suíte server/__tests__/campanhas-whatsapp.test.ts (152 te*